I've identified several potential performance issues across both the backend and frontend:

## Backend Performance Issues

### 1. **N+1 Query Problem in `list_students` (teacher.py, lines 333–384)** Not an Issue
The announcement classrooms aggregation uses a subquery, but the final loop still iterates through results without fully materializing relationships. The classroom names are parsed from a string result, which works but misses opportunities for eager loading optimization.

**Issue:** Lines 370–383 iterate and parse strings when related data was already fetched.

### 2. **Multiple Sequential Queries in Roster Operations (roster.py)** ClaudeFixed
In `find_and_link_teacher_code()` (lines 160–178), there are separate database roundtrips for:
- Fetching classroom IDs for a student
- For each classroom ID, inserting into `classroom_students`
- Fetching parent IDs for the placeholder
- For each parent ID, inserting into `parent_students`

**Performance Hit:** On line 163, each classroom's insertion is a separate `await db.execute()` call. With many classrooms per student, this serializes I/O.

**Fix:** Batch these operations into bulk INSERT statements instead of looping.

### 3. **Repeated Database Lookups in `import_classrooms_pdf` (teacher.py, lines 230–316)** ClaudeFixed
Line 285 calls `_find_duplicate_school_ids()` for every page in the PDF, even though duplicate checking could be cumulative within a single import session.

**Issue:** If a 10-page PDF is imported, duplicate checking happens 10 times against an ever-growing `seen_school_ids` set.

### 4. **Inefficient Announcement Readers Query (teacher.py, lines 482–532)** ClaudeFixed
Lines 502–518:
- Query all students in target classrooms
- Fetch all reads for that announcement
- Manual in-Python split into readers/non-readers

For announcements with thousands of student enrollments, loading all students into memory then splitting in Python is wasteful. This should be two focused queries (read vs. not-read).

### 5. **Missing Pagination in `get_announcement_readers()` (teacher.py)** ClaudeFixed
Returns all readers/non-readers without pagination. A teacher with 1000+ students enrolled won't scale.

### 6. **Unindexed or Poorly Optimized Joins** ClaudeFixed
The roster matching in `find_and_link_teacher_code()` (lines 128–140) joins `StudentProfile` → `teacher_students` → classroom/grade relationships. Without indexes on foreign keys or on `(user_id, school_id)` tuples, this query can be slow for large rosters.

## Frontend Performance Issues

### 7. **No Connection Pooling or Request Batching (api_client.dart)** ClaudeFixed
Each API call is independent. For parent home screen with multiple simultaneous API requests (list students + load announcements + fetch filters), there's no HTTP/2 connection reuse or batching optimization.

### 8. **Synchronous Locale Loading (main.dart, line 50)** ClaudeFixed
```dart
```

await loadSavedLocale();
```

If SharedPreferences reads are slow, this blocks app startup. Consider non-blocking initialization.

### 9. **Unoptimized State Management (parent_home_screen.dart)** Not an Issue, Another Issue is ClaudeFixed
Lines 32–44: `_loadStudents()` is `Fire-and-forget` in initState. If it fails silently and reruns on every rebuild, multiple redundant API calls happen.

### 10. **Announcement Read Tracking Inefficiency (fcm_service.dart, lines 60–65)** Not an Issue
Token registration on every token refresh and init without deduplication. If FCM token refresh triggers frequently (network changes, app restart), redundant `/student/device-token` POST calls accumulate.

### 11. **Large String Parsing in Announcement Lists (student.py, lines 180–191 & Not an Issue
parent_home_screen.dart)**
Classroom names are returned as comma-separated strings and split in the UI. For a student in 10+ classrooms per announcement, string parsing scales poorly. Use a structured list format instead.

## Quick Wins In short: 1 is wrong, 2 is mostly done, 3 is already done, 4 is unnecessary and counterproductive, 5 is real but minor. The only thing worth considering is #5. So #5 is ClaudeFixed

1. **Add indexes** to `(teacher_id, school_id)` in `student_profiles` and `(announcement_id, student_id)` in `announcement_read`.
2. **Batch roster operations** with a single `INSERT ... ON CONFLICT` for all classroom/parent links.
3. **Paginate `/announcements/{id}/readers`** response.
4. **Cache announcement readers** with a 5-minute TTL rather than recomputing on every request.
5. **Add **request deduplication** in FCM token registration—skip if the token hasn't changed.
```